

TALLINN UNIVERSITY OF TECHNOLOGY
IT College

Marko Linde 176292IADB

CALCULATOR

Javascript (ICD0006)

Instructor: Andres Käver

Tallinn 2022

Table of Contents

1 Assignment	3
2 Overview	4
2.1 Calculator logic.....	4
2.2 Non-functional requirements	5
2.3 Display	6
2.4 Functionality	7
2.4.1 Main.....	7
2.4.2 Evaluations after calculate command	7
2.4.3 Calculation command	7
2.5 Dependancies.....	8
Annex 1 – code	9

1 Assignment

Write simple calculator using Bootstrap (or any other css lib) based design and simple js functionality.

Provide basic functions - +, -, *, /, C

Write your calculator using an object oriented approach. Separate code between business logic and ui.

Please implement classical calculator UI - like you have in your mobile phone.

Nice to have - calculator is done in responsive design. Fits full-screen, both on desktop and on mobile.

Use modules/exports/imports.

UI has to be generated fully in JS.

Only allowed element inside <body> tag is bootstrap import and single custom js import.

Support multiple calculators on page in parallel (every calculator is separate from each other).

Implement UI for dynamic adding and removal

Use webpack, babel and typescript.

Type any is not allowed. Casting is not allowed - use type guards.

2 Overview

index.css taken from calculator example . Index.js , css and html is written or modified by document author.

2.1 Calculator logic

Basic idea comes from logic, that the functionality of the calculator can be expanded.

- All calculator clicks are saved in a long string only array which allows:
 - to display the entire equation to the user



Drawing 1. Equation view

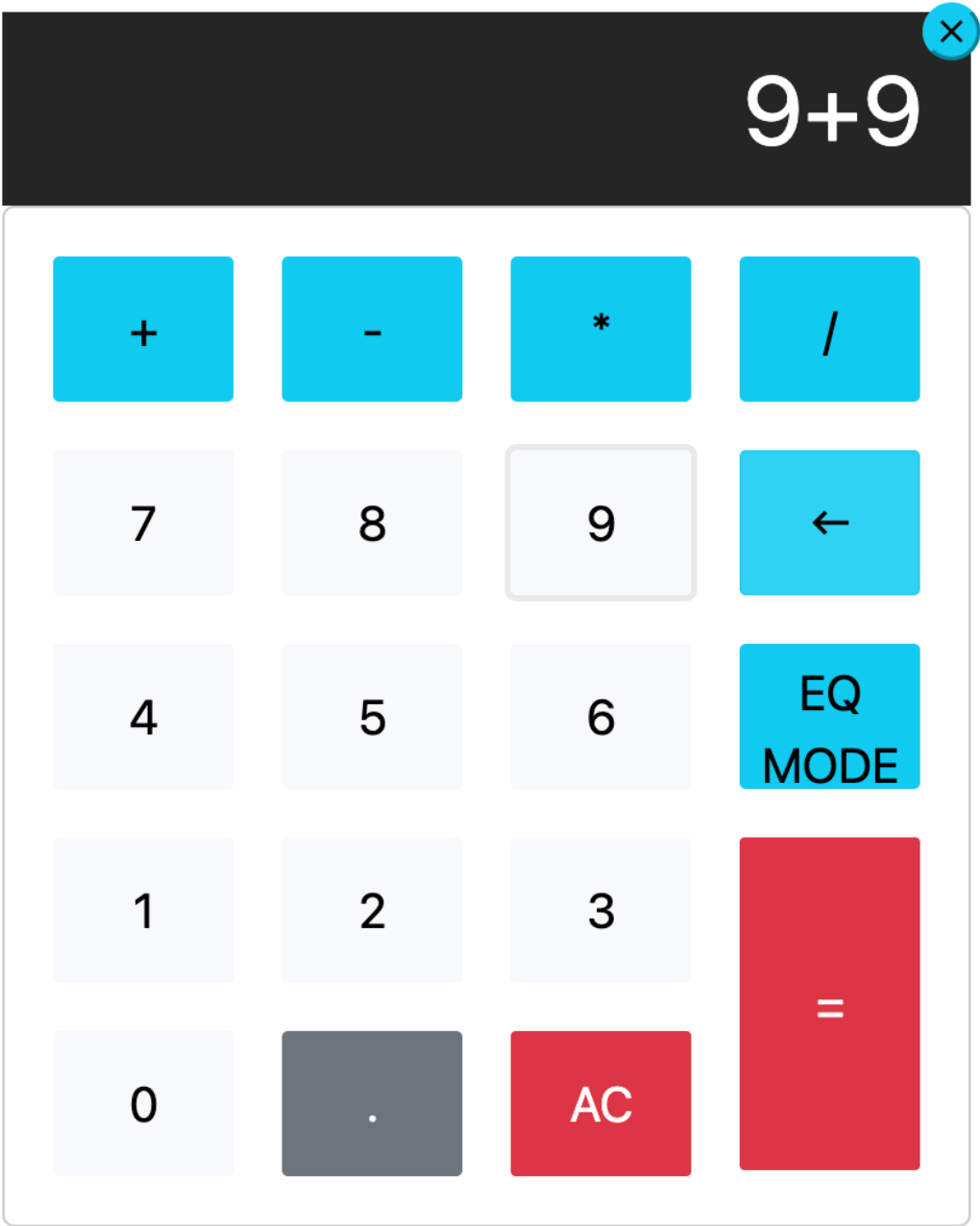
- to add undo button functionality
 - to add calculate only parts of the equation functionality
 - to add click on equation element and repair it directly functionality
 - to make calculator more scientific
- Calculator starts to calculate only when clicking the equal sign
- In EQ mode the calculations are done from left to right with rule: first multiply or divide then add or subtract
- In Not EQ mode the calculations are done from left to right
- Backspace removes last equation entry

- X mark closes calculator
- Calculator business logic with calculator module are injected into Calculator UI
- Index.html is segregated into navbar and main sections
- Calculator receives distinct ID (differentiates calculators) by card1.....n
- ID is generated by static class (id is not needed, but in case required later, it is kept)
- Close button is inside calculator module
- Calculator add but in navbar.

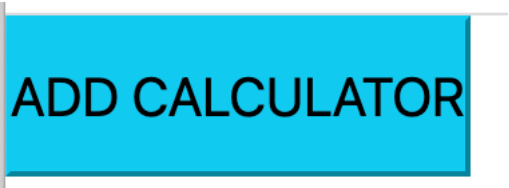
2.2 Non-functional requirements

- is in typescript for less errors and code breaking, strongly defined variables, makes code more readable
- type guards for narrowing down the scope for type

2.3 Display



Drawing 2. Calculator screen



Drawing 3. Add calculator button

2.4 Functionality

2.4.1 Main

- Elements are observed through node lists
- Html calculator is generated by module class
- Every button click value is inserted into equations array separately
- Equal button clicking calls calculation function
- AC button click calls overwriting of the previous array by empty array
- Backspace removes last equation entry
- EQ mode switches between two calculation modes
- X button closes calculator
- Add button on navbar adds calculator

2.4.2 Evaluations after calculate command

- Operators the start (except minus) and end equation are evaluated – that they don't exist
- Two operators straight are not allowed
- Two dots straight are not allowed
- Operator and dot are not allowed

2.4.3 Calculation command

- Number of operators is searched
- First calculation operator is searched from the equation and its index is returned.
- All numbers between operators are converted to full numbers
- Calculated value is inserted back into the equation
- New calculation command iteration will begin

2.5 Dependancies

- css-loader for compiling css to js
- style-loader for style injecting to DOM
- webpack as js file bundler
- babel for making current typescript js compatible

Annex 1 – code

```
"use strict";
import './index.css';
import { CalculatorModule } from "../public/calculatorModule";
import { CalculatorUI } from "../public/calculatorUI";
import { Calculus } from "../public/calculus";

/**
 * Setting up WebPage areas
 */
let navBar = document.createElement('div');
let navBarButton = document.createElement('button');
navBarButton.textContent = 'ADD CALCULATOR';
navBarButton.classList.add('btn', 'btn-info');

navBar.appendChild(navBarButton);
document.body.appendChild(navBar);

let centerArea = document.createElement('div');

document.body.appendChild(centerArea);

function addCalculator(){
    navBarButton.onclick = function() {
        let calcMod = new CalculatorModule();
        centerArea.appendChild(calcMod.calculatorContent);
        new CalculatorUI(new Calculus(), calcMod);
    }
}

addCalculator();
```

```

<html>
  <head>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-
1BmE4kWBq78iYhFldvKuhfTAU6auU8tT94WrHftjDbrCEXSU1oBoqyl2QvZ6jIW3" crossorigin="anonymous">
    <link href="https://fonts.googleapis.com/icon?family=Material+Icons" rel="stylesheet">
  </head>
  <body>
</body>
</html>

```

```

/**
 * Keeps distinct id-s.
 */
export class ModuleId {
  static idCounter : number = 0;
  static freeIds : number[] = [];

  static getNextId() :number {
    if (this.freeIds.length > 0) return this.freeIds.pop()!;
    else return ++this.idCounter;
  }

  static freeId(id:number) {
    this.freeIds.push(id);
  }
}

```

```

/**
 * Equation is class to preserve user clicked
 * information and perform basic operations with
 * array equation
 */
export class Equation {

    equation: string[] = [];
    number: string[] = [];

    constructor(){
        this.equation = [];
    }

    clear(){
        this.equation = [];
        this.number= [];
        console.log(JSON.stringify(this.equation));
    }

    addElement = (elem:string) => {
        this.equation.push(elem);
        console.log(JSON.stringify(this.equation));
    }

    // all individual numeric elements between symbols are converted to full numbers
    convertEquationToFullNumbers = (operators:string[]) =>{
        let index: number = 0;
        let number: string = "";
        let compressedEquation: string[] = [];

        this.equation.forEach(elem =>{
            if (this.getIndex(operators,elem) > -1 && !(elem === '-' && index === 0)){
                compressedEquation.push(number);
            }
        });
    }
}

```

```

        compressedEquation.push(elem);
        number="";
    } else {
        if (number === "") number = elem;
        else number = number.concat(this.equation[index]);
    }
    index++;
});
if (number !== ""){
    compressedEquation.push(number);
}

this.equation = compressedEquation;
console.log(this.equation);
}

getNumberOfSymbolsInEquation = (operators:string[]) =>{
    let counter = 0;

    this.equation.forEach(elem => {
        if (this.getIndex(operators,elem) > -1) counter +=1;
    });

    return counter;
}

getIndex(operators:string[],sign:string) :number{
    return operators.findIndex((element) => element === sign);
}

getConcatEquation = () :string =>{
    return this.equation.join('');
}

```

```

getIndexValue = (index:number) :string => {
    return this.equation[index];
}

getPreviousValue = (index:number) :string => {
    return this.equation[index-1];
}

getNextValue = (index:number) :string => {
    return this.equation[index+1];
}

changeEquation = (index:number,value:string) =>{
    if (index >= 0) this.equation.splice(index - 1,3,value);
}

removeLast = () =>{
    this.equation.pop();
}
}

```

```

import { Equation } from "./equation";
/**
 * Businesslogic class to perform calculations and find errors
 * Todo: move errors to separate class
 */
export class Calculus {

    equation: Equation;
    equationState: string | undefined;
    operators: string[];
    eqMode: number;

```

```

constructor(){
  this.equation = new Equation();
  this.equationState = undefined;
  this.operators = ['+', '-', '*', '/'];
  this.eqMode = 0;
}

actionCaller = (action:string) =>{
  console.log(action);
  console.log(JSON.stringify(this.equation));

  if (!(this.findErrorAction(action))){
    switch (action) {
      case 'AC':
        this.equation.clear();
        break;
      case '=':
        this.equation.convertEquationToFullNumbers(this.operators);
        this.calculate();
        break;
      case 'keyboard_backspace':
        this.equation.removeLast();
        break;
      case 'EQ MODE':
        this.eqMode = this.setEqMode();
        console.log("Eqmode :" + JSON.stringify(this.eqMode));
        break;
      default:
        this.equation.addElement(action);
    }
  }
  this.equationState = this.equation.getConcatEquation();
}

```

//switches system between mathematical standard and linear calculation modes

```

setEqMode = () : number =>{
    return (this.eqMode === 1) ? 0 : 1;
}

//returns equation to screen
getEquationState = () : string=>{
    return (typeof this.equationState === 'string') ? this.equationState! : 'Error';
}

//find user entered problems
findErrorAction = (action:string) : number =>{
    let error = 0;
    if (this.equation.getConcatEquation().length === 0 && this.equation.getIndex(this.operators,action) > -1 &&
        action !== '-') error = 1;
    else if ((this.isPreviousEntryAnOperator(this.operators) || this.isPreviousEntryAnOperator(['.'])) &&
        action === '=') error = 1;
    else if ((this.isPreviousEntryAnOperator(this.operators) || this.isPreviousEntryAnOperator(['.'])) &&
        (this.equation.getIndex(this.operators,action) > -1 || action === '.')) error = 1;
    return error;
}

isPreviousEntryAnOperator = (operators:string[]) =>{
    return (this.equation.getIndex(operators,
        this.equation.equation[this.equation.equation.length-1]) > -1) ? 1 : 0;
}

calculate = () =>{
    while(this.equation.getNumberOfSymbolsInEquation(this.operators)){
        if (this.eqMode === 1) this.calculateIndexValue(this.getNextCalculationIndex());
        else this.calculateIndexValue(1);
    }
}

//calculate equation value on index operator with before and after numbers
//and inserts subcalculation back to equation

```

```

calculateIndexValue = (startIndex:number) =>{
  let value = NaN;

  switch(this.equation.getIndexValue(startIndex)) {
    case '*':
      value = parseFloat(this.equation.getPreviousValue(startIndex)) *
        parseFloat(this.equation.getNextValue(startIndex));
      break;
    case '/':
      value = parseFloat(this.equation.getPreviousValue(startIndex)) /
        parseFloat(this.equation.getNextValue(startIndex));
      break;
    case '+':
      value = parseFloat(this.equation.getPreviousValue(startIndex)) +
        parseFloat(this.equation.getNextValue(startIndex));
      break;
    case '-':
      value = parseFloat(this.equation.getPreviousValue(startIndex)) -
        parseFloat(this.equation.getNextValue(startIndex));
      break;
    default:
      value;
  }
  console.log('Index calculation value: ' + value);
  this.equation.changeEquation(startIndex,value.toString());
  console.log('Equation: ' + JSON.stringify(this.equation.equation));
}

//based on mathematical logic get first calcualtion operator
getNextCalculationIndex = ( ) :number =>{
  let startIndex = NaN;

  if (((this.getEquationIndex('*') !== -1) && (this.getEquationIndex('*') < this.getEquationIndex('/')) ||
    ((this.getEquationIndex('/') === -1) && (this.getEquationIndex('*') > this.getEquationIndex('/'))))
    startIndex = this.getEquationIndex('*');

```



```

        else if (this.getEquationIndex('/') >= 1) startIndex = this.getEquationIndex('/');
        else if (this.getEquationIndex('+') >= 1) startIndex = this.getEquationIndex('+');
        else if (this.getEquationIndex('-') >= 1) startIndex = this.getEquationIndex('-');

        console.log('Starting with index: ' + startIndex);
        return startIndex;
    }

    getEquationIndex = (value:string) :number =>{
        return this.equation.getIndex(this.equation.equation,value);
    }
}

```

```

import { CalculatorModule } from "./calculatorModule";
import { Calculus } from "./calculus";
/**
 * Takes care of calculator user interface, which
 * relays the information from web page to calculation
 * and pack
 */
export class CalculatorUI{

    calculus: Calculus;
    module: CalculatorModule;

    constructor(calculus:Calculus,module:CalculatorModule){
        this.calculus = calculus;
        this.module = module;

        module.closeKey.onclick = (() => {
            this.module.freeModuleId();
            this.module.calculatorContent.remove();
        });
    }
}

```

```

    Array.from(module.calculatorKeyModule.children).forEach(child =>{
      if (child instanceof HTMLButtonElement){
        child.onclick = (() => {
          this.action(child.getAttribute('value')!);
          if (child.getAttribute('value') === 'EQ MODE'){
            child.style.color= (child.style.color === 'red') ? 'black' : 'red';
          }
        });
      }
    });
  });
}

action = (action:string) => {
  this.calculus.actionCaller(action);
  this.refresh();
}

//refreshes display
refresh = () => {
  this.module.calculatorScreen.value = (typeof this.calculus.getEquationState() === undefined) ?
    'Invalid value' : this.calculus.getEquationState()!;
}

}

```

```

import { ModuleId } from "../moduleId";
/**
 * Class which groups together all html elements for calculator
 */
export class CalculatorModule{

    calculatorContent: HTMLDivElement;
    calculatorScreen: HTMLInputElement;
    calculatorKeyModule: HTMLDivElement;
    closeKey: HTMLButtonElement;
    moduleId: number;

    constructor(){
        this.moduleId=ModuleId.getNextId()!;
        this.calculatorContent = document.createElement("div");
        this.calculatorContent.className = "calculator card"+this.moduleId;
        this.calculatorContent.style.display = 'inline-block';

        this.closeKey = this.calculatorKey('close','material-icons','btn-info','button-close')
        this.calculatorContent.appendChild(this.closeKey);

        this.calculatorScreen = this.setCalculatorScreen("calculator-screen","z-depth-1");
        this.calculatorContent.appendChild(this.calculatorScreen);

        this.calculatorKeyModule = this.setCalculatorKeyModule();
        ['+', '-', '*', '/'].forEach(operator =>{
            this.calculatorKeyModule.appendChild(this.calculatorKey(operator, 'btn-info', 'btn'));
        });
        ['7', '8', '9'].forEach(operator =>{
            this.calculatorKeyModule.appendChild(this.calculatorKey(operator, 'btn-light', 'waves-effect', 'btn'));
        });
        this.calculatorKeyModule.appendChild(this.calculatorKey('keyboard_backspace', 'operator', 'material-icons', 'btn-info', 'btn'));
        ['4', '5', '6'].forEach(operator =>{
            this.calculatorKeyModule.appendChild(this.calculatorKey(operator, 'btn-light', 'waves-effect', 'btn'));
        });
    }

```

```

    this.calculatorKeyModule.appendChild(this.calculatorKey('EQ MODE','operator','btn-info','btn'));
    ['1','2','3','0'].forEach(operator =>{
        this.calculatorKeyModule.appendChild(this.calculatorKey(operator,'btn-light','waves-effect','btn'));
    });
    this.calculatorKeyModule.appendChild(this.calculatorKey('.', 'decimal', 'function', 'btn-secondary', 'btn'));
    this.calculatorKeyModule.appendChild(this.calculatorKey('AC', 'all-clear', 'function', 'btn-danger', 'btn-sm', 'btn'));
    this.calculatorKeyModule.appendChild(this.calculatorKey('=', 'equal-sign', 'operator', 'btn-danger', 'btn-sm', 'btn'));

    this.calculatorContent.append(this.calculatorKeyModule);
}

freeModuleId(){
    ModuleId.freeId(this.moduleId);
}

setCalculatorScreen(...cssClasses:string[]) : HTMLInputElement {
    let calculatorScreen = document.createElement("input");
    calculatorScreen.type = "text";

    cssClasses.forEach(cssClass => {
        calculatorScreen.classList.add(cssClass);
    });

    calculatorScreen.value = "";
    calculatorScreen.disabled = true;
    return calculatorScreen;
}

setCalculatorKeyModule() : HTMLDivElement{
    let calcModule = document.createElement("div");
    calcModule.className = "calculator-keys"
    return calcModule;
}

calculatorKey(text:string,...cssClasses:string[]) : HTMLButtonElement{

```

```

    let button = document.createElement("button");

    cssClasses.forEach(cssClass => {
      button.classList.add(cssClass);
    });

    button.value = text;
    button.textContent = text;
    return button;
  }
}

```

```

/*webpack configuration */
const path = require('path')
const HtmlWebpackPlugin = require('html-webpack-plugin')
const { Template } = require('webpack')

module.exports = {
  mode: 'production',
  entry: './src/index',
  output: {
    path: path.resolve(__dirname, 'dist'), /*current directory with joined dist*/
    filename: "app.js" /*takes name from entry and make js file */
  },
  resolve: {
    extensions: ['.ts', '.tsx', '.js', '.json', '.css']
  },
}

```

```

plugins : [
  new HtmlWebpackPlugin(
    {
      template : "./src/index.html",
      inject : "body", /*inject into body tag*/
      minify : false,
    }
  )
],
/*https://www.npmjs.com/package/css-loader*/
module: {
  rules: [
    {
      test: /\.css$/i,
      use: ["style-loader", "css-loader"],
      exclude: /\.module\.css$/,
    },
    {
      test: /\.?(ts|js)x?$/,
      use: 'babel-loader',
      exclude: /node_modules/,
    },
  ],
},
},
}

```